.NET Core vs .NET Framework Code Execution Process

Back to: ASP.NET Core Web API Tutorials

.NET Core vs .NET Framework Code Execution Process

When we build and run applications using the .NET platform, the source code doesn't get executed directly by the operating system. Instead, it goes through a multi-step process involving compilation, conversion to an intermediate language, and final execution as machine code.

Both **.NET Framework** and **.NET Core** follow this approach, but they differ in how the components are designed and executed. While the .NET Framework is Windows-only and tightly integrated with the OS, **.NET Core** is modern, **open-source**, **cross-platform**, and designed to meet today's needs, such as cloud, microservices, and containers.

Now, we will understand the **step-by-step code execution process** in both .NET Framework and .NET Core, understanding the key differences.

.NET Framework Code Execution Process:

When we write and run a **.NET Framework application**, the source code does not immediately become **native machine code** (like in C or C++). Instead, the .NET Framework uses a **multi-stage execution pipeline** involving:

- **Language-specific compilers** (C#, VB.NET, F#)
- **Intermediate Language (IL / MSIL)**
- **Metadata**
- **The Common Language Runtime (CLR)** with its **JIT Compiler**
- **Runtime services** like Garbage Collection, Security, and Exception Handling

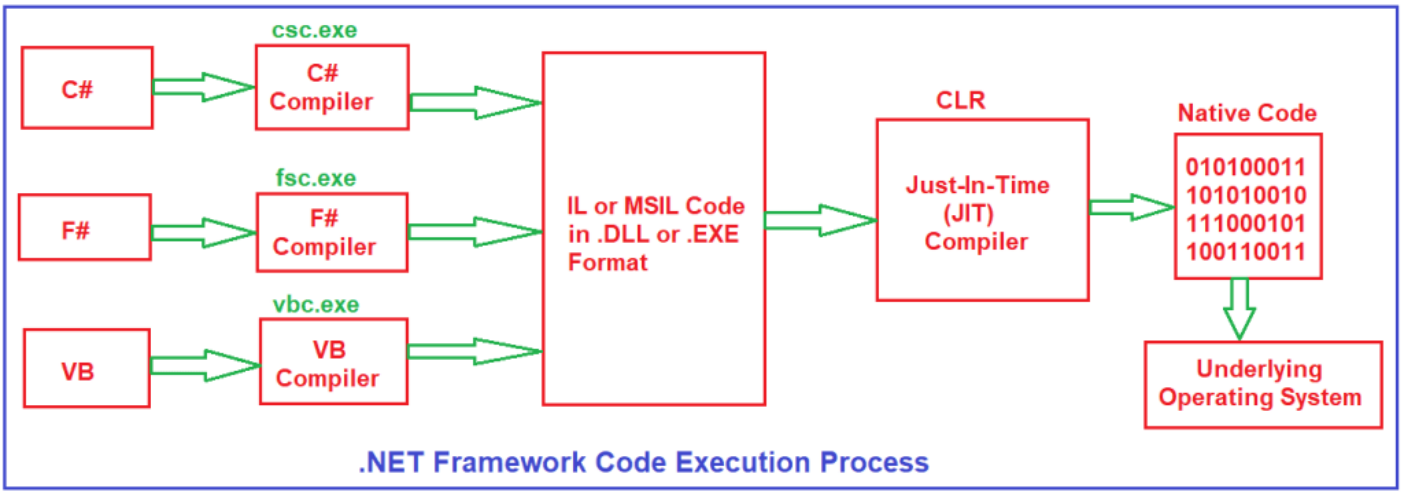
This design provides **three key benefits**:

1. **Language Interoperability** → multiple languages (C#, VB, F#) can work together.
2. **Platform Independence (within Windows Ecosystem)** → IL is CPU-agnostic and portable.
3. **Runtime services** → CLR manages memory, security, and execution.

Let us first discuss the .NET Framework Application Execution Process. The code execution process of a .NET Framework Application includes the following four steps.

1. **Choosing a Compiler**
2. **Compiling Source Code to MSIL**
3. **Compiling MSIL to Native Code**
4. **Running Native Code**

For a better understanding, please have a look at the following diagram.



Step 1: Choosing a Compiler

The .NET Framework is a **Language-Independent Platform**. Developers can write applications in different languages, but eventually, they all run on the **Same Runtime (CLR)**. Each programming language has its own compiler. Depending on the language we choose, the respective language compiler will compile the source code:

- **C# Source Code (.cs files)** → Compiled using **CSC (csc.exe)**.
- **VB.NET Source Code (.vb files)** → Compiled using **VBC (vbc.exe)**.
- **F# Source Code (.fs files)** → Compiled using **FSC (fsc.exe)**.

Each compiler has two responsibilities:

1. Convert **high-level language constructs** (loops, classes, methods) into **IL instructions**.
2. Generate **metadata** about the compiled code (types, methods, assemblies, etc.).

Why this design?

- It allows developers to pick the language they are most comfortable with.
- Allows different languages to target the **same runtime**
- Regardless of the language, the compiler produces a **common IL format**.
- This enables **language interoperability** → for example, a class written in C# can be consumed in VB.NET or F#.
- Promotes consistency across the .NET ecosystem.

Real-world analogy: Think of C#, VB.NET, and F# as people speaking different languages. Each has its own translator (compiler) who translates its words into a **common language (IL)**. The common listener (CLR) can then understand and execute it.

Step 2: Compiling Source Code to IL (MSIL)

Once the compiler processes the source code, it produces:

1. **IL Code (Intermediate Language / Microsoft Intermediate Language):**
   - CPU-independent instructions that define program logic.
   - Example IL instructions: ldstr, add, call, brtrue, etc.
   - Platform-neutral → can run anywhere with a CLR implementation.
2. **Metadata:**
   - Contains details about:
     - Types (classes, interfaces, structs)
     - Members (methods, fields, properties)
     - Assembly references (dependencies on other DLLs/assemblies)

What is an Assembly Reference?

- It's like a pointer inside the compiled assembly (.EXE or .DLL) telling the CLR which external libraries or components this code depends on.
- **Example:** If your project uses System.Data.SqlClient, your assembly will contain a reference to System.Data.dll.

Output Files

- **Executable (.EXE)** → If you build a Console App, WPF, or WinForms project.
- **Library (.DLL)** → If you build a Class Library or ASP.NET Web App.

Both EXE and DLL files in .NET contain **IL (Intermediate Language) and metadata**, not native machine code.

ILDASM Tool (Intermediate Language Disassembler)

- A diagnostic tool provided by Microsoft.
- Opens an EXE/DLL file and displays IL instructions and metadata.
- Helps developers **inspect the compiled IL** and verify what the compiler produced.

Why IL Instead of Direct Machine Code?

- **Portability:** IL runs on any platform that has a CLR implementation.
- **Security:** CLR verifies IL before execution, preventing unsafe operations.

- **Optimization:** Native code is produced at runtime, designed for the executing CPU (x86, x64, ARM).

This means **one IL binary can adapt to multiple environments**.

Step 3: Compiling IL to Native Code (JIT Compilation)

When the program runs, the **CLR (Common Language Runtime)** loads the IL code into memory. At this point, the **JIT (Just-In-Time) Compiler** comes into play.

The JIT Compiler

- Converts IL into **Native Machine Code** (CPU instructions).
- This translation happens **at runtime**, just before the method is executed.
- **Example:** If you run the same IL code on Windows with an Intel CPU vs. Windows with an ARM CPU, JIT generates **different machine code**, optimized for each hardware.

Advantages of JIT

- **Runtime optimization:** JIT can inline methods or optimize loops depending on the hardware.
- **Cross-hardware adaptability:** Same IL runs efficiently on Intel, AMD, or ARM processors.
- **Reduced memory footprint:** Only compiles what is needed during execution.

Analogy: Think of IL as a universal recipe written in symbolic form. JIT acts like a local chef, translating the recipe into native cooking instructions suitable for the available kitchen (hardware/OS).

Types of JIT

The JIT (Just-In-Time) Compiler is a part of the CLR (Common Language Runtime). Its role is to convert Intermediate Language (IL/MSIL) into native machine code when the application runs. There are three types of JIT compilation approaches:

Normal JIT

- Compiles a method **only when it is called for the first time**.
- After that, the native code is **kept in memory** so it runs faster next time.
- Used by default in **most .NET Framework applications**.
- If you have 100 methods but only call 20, then only those 20 are compiled.

Analogy: Like cooking only when you're hungry. You don't cook every recipe in your kitchen — you prepare meals only when needed.

Econo JIT

- Also compiles when the method is called, **but does not keep it in memory**.
- If the method is needed again, it gets compiled again.
- Saves memory but makes performance slower.
- Used in **older .NET Compact Framework** (for small devices), not in modern apps.

Analogy: Like renting a book from a library instead of buying it. You free up space when you return it, but if you need it again, you must borrow it again.

Pre-JIT (NGen.exe)

- The **entire program (all methods)** is compiled into native code **before running**.
- Not default, must be done manually with **ngen.exe**.
- Makes startup faster (no JIT delay at runtime).
- Used in large applications (like Office, big enterprise apps).

Analogy: Like preparing all meals in the morning and storing them in the fridge. Meals are ready instantly, but may not be as fresh or as well-tasted as when cooked on demand.

Step 4: Running the Native Code

After the IL has been converted into **native code**, the **Operating System executes it directly**. However, the **CLR remains active**, providing crucial runtime services, such as:

Memory Management & Garbage Collection

- CLR automatically allocates memory for objects on the **managed heap**.
- The **Garbage Collector (GC)** periodically frees memory for unused objects.
- Prevents memory leaks that were common in languages like C/C++.

Security

- CLR enforces **Code Access Security (CAS)** to prevent unauthorized access.
- Ensures assemblies run within permissions granted (e.g., restricted IO access in sandboxed environments).
- Provides **type safety** to prevent unsafe operations (like buffer overflows).

Exception Handling

- CLR provides a **standardized model** for handling errors (try, catch, finally) across all .NET languages.
- Prevents application crashes by handling runtime errors gracefully.

Debugging & Profiling

- CLR supports **cross-language debugging** (e.g., stepping into VB code from a C# project).
- Profiling APIs enable developers to identify performance bottlenecks and optimize memory usage.

BCL and FCL in .NET Framework:

Framework Class Library (FCL)

- Complete set of libraries for UI, Data Access, Networking, XML, etc.

Base Class Library (BCL)

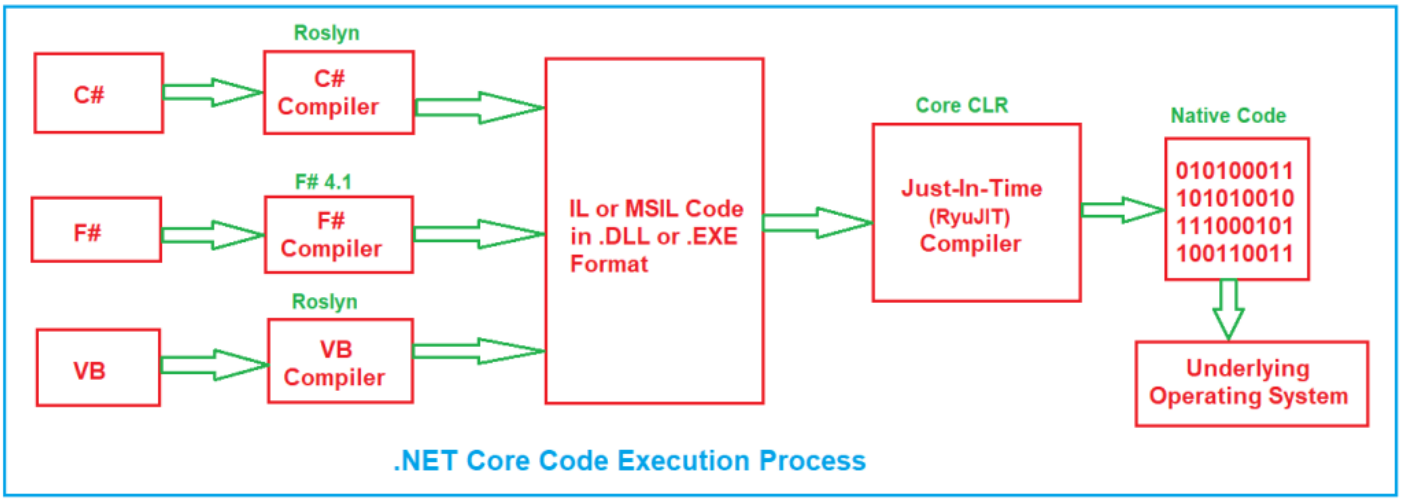
- A **subset of FCL**.
- Includes fundamental classes:
  - Collections (List, Dictionary)
  - File I/O (File, Stream)
  - Data types (String, Int32)
  - Threading & Tasks

Together, **Compiler + CLR + BCL + FCL = Core Execution Model of .NET Framework**.

ASP.NET Core Code Execution Process:

Now, let us see how .NET Core Code Execution takes place compared to the .NET Framework. The **.NET Core application execution process** is similar in principle to the **.NET Framework**, but it introduces **modernized, lightweight, and cross-platform components**.

Whereas the .NET Framework was **Windows-only and monolithic**, .NET Core was **re-engineered** to be **modular, open-source, and cross-platform**, making it suitable for **cloud-native and containerized environments**. For a better understanding, please refer to the following diagram.



Step 1: Choosing a Compiler in .NET Core

Like the .NET Framework, .NET Core allows development in multiple languages. Each programming language has its own compiler:

- **C#** → Compiled using **Roslyn C# compiler (csc.dll)**
- **VB.NET** → Compiled using **Roslyn VB compiler (vbc.dll)**
- **F#** → Compiled using the **F# compiler (fsc.exe)**

Roslyn: The Big Difference

- Unlike the old **csc.exe/vbc.exe** in .NET Framework, **Roslyn** is **modern, extensible, and open-source**.
- Provides **Compiler-as-a-Service (CaaS)**:
  - It doesn't just compile → it also exposes APIs for:
    - Syntax tree generation
    - Semantic analysis
    - Real-time diagnostics (squiggly red lines in Visual Studio/VS Code)
    - Code refactoring tools
    - IntelliSense Support
- This makes Roslyn **both a compiler and a code analysis platform**.
- IDEs like Visual Studio and VS Code offer **rich editing features** in real-time.

Analogy:

- In .NET Framework, compilers were like "Black Box Translators" → you gave them code, and they just gave you IL.
- In .NET Core, Roslyn is like a "Translator + Teacher" → it translates your code but also helps you understand and improve it.

Step 2: Compiling Source Code to IL (MSIL)

When you build the project, the compiler translates the source code into:

1. **IL (Intermediate Language / MSIL)**
   - CPU-independent instructions.
   - Same IL can run on **Windows, Linux, or macOS**, as long as CoreCLR is present.
2. **Metadata**
   - Describes everything in your code:
     - Types (classes, structs, interfaces)
     - Members (methods, properties, fields)
     - **Assembly references** (which other DLLs your code depends on).

- Makes assemblies **self-describing** (reflection and tools like ILSpy work because of metadata).

Output Files

- **.EXE** → If it's a console application.
- **.DLL** → If it's a class library or ASP.NET Core app.

Managed Code in .NET Core

This IL is called **Managed Code** because it is executed under the supervision of the **CoreCLR runtime**, which handles:

- Memory management
- Security enforcement
- Garbage collection
- Threading

In .NET Core, just like .NET Framework, tools like **ILSpy, dotPeek, and dnSpy** allow you to open .NET Core DLL/EXE and view IL + Metadata.

Step 3: Compiling IL to Native Code

When you run the application, the .NET Core runtime (Core CLR) loads the IL into memory. Then the **JIT Compiler** (called **RyuJIT**) converts IL into **native machine code**.

RyuJIT Features (Modern JIT)

- **Faster & more efficient** than old JIT.
- Optimized for **64-bit architectures** from the ground up.
- Supports **new CPU instructions (SIMD, AVX, etc.)** for high-performance workloads.
- Produces better-optimized machine code than the legacy JIT used in the .NET Framework.

Types of Code Execution in .NET Core

1. **Normal JIT Execution**
   - IL is compiled into native code **the first time a method is called**.
   - Code is cached in memory → reused on later calls.
   - Similar to .NET Framework's JIT.
2. **ReadyToRun (R2R) Execution**
   - IL can be **precompiled into native code at publish time** using tools like crossgen or the dotnet publish –readytorun option.
   - Stored in assemblies as **R2R images**.

- Reduces **startup time** (especially in microservices and containerized apps).
- Better than **NGen** in .NET Framework because:
  - It's cross-platform.
  - It works well with modern runtime optimizations.

Analogy:

- **Normal JIT:** Cook a dish only when an order is placed.
- **ReadyToRun:** Pre-cook popular dishes in the morning so they're ready to serve faster later.

Step 4: Running the Native Code

Once IL is converted to native machine code, the **Operating System executes it directly**, while the **CoreCLR provides runtime services**, including:

- **Automatic Memory Management & Garbage Collection (GC):**
  - Allocates objects on the managed heap.
  - Reclaims unused memory automatically.
- **Security Services:**
  - Ensures type safety and prevents unsafe memory access.
  - Provides sandboxing for restricted environments.
- **Debugging & Diagnostics:**
  - Allows cross-platform debugging (Windows, Linux, macOS).
  - Rich diagnostic tools like dotnet-trace, dotnet-dump, and EventPipe.
- **Exception Handling:**
  - Consistent across languages (try/catch/finally).
  - Prevents system crashes from unhandled exceptions.

Key Changes in .NET Core vs .NET Framework:

If you observe, the execution process remains the same. Only a few components are changed when compared with the .NET Framework.

Compilers

- **.NET Framework:** CSC, VBC, FSC (traditional compilers).
- **.NET Core:** Roslyn for C# & VB (open-source, modern), F# compiler (FSC) updated for CoreCLR.

Class Libraries

- **.NET Framework:** Framework Class Library (FCL), Base Class Library (BCL).

- **.NET Core: CoreFX Libraries** → modular, lightweight, NuGet-based.
  - Developers add only what they need.
  - Fully cross-platform (Windows, Linux, macOS).

Runtime (CLR vs CoreCLR)

- **.NET Framework:** CLR (Windows-only, monolithic).
- **.NET Core:** CoreCLR (modular, cross-platform, open-source).
  - Supports modern OS features.
  - Runs on **Windows, Linux, macOS, Docker**.

JIT Compiler

- **.NET Framework:** Legacy JIT (x86, x64) + limited optimizations.
- **.NET Core:** RyuJIT (modern, optimized for 64-bit, cross-platform).

Why Reimplement Components?

Now, the obvious question that should come to mind is why we need to reimplement all these components that are already available in the .NET framework. So, the answer is the same as why Microsoft implemented .NET Core: it is for Open-Source and Cross-Platform.

- **CLR** → **CoreCLR** → Needed cross-platform runtime.
- **FCL** → **CoreFX** → Lighter, modular, open-source libraries.
- **CSC** → **Roslyn** → Extensible, compiler-as-a-service, open-source.
- **JIT** → **RyuJIT** → Modern, unified, 64-bit optimized compiler.

While the overall code execution flow, from source code to native machine code, is similar in both the .NET Framework and .NET Core, the internal components in .NET Core have been redesigned to be faster, more modular, and cross-platform. Tools like **Roslyn**, **CoreCLR**, **CoreFX**, and **RyuJIT** make .NET Core more flexible, lightweight, and suitable for modern development scenarios, such as cloud-based applications and microservices.

These improvements not only enhance performance and portability but also provide developers with more control and increased productivity. Understanding these execution internals allows developers to build better-performing, scalable, and maintainable applications on the .NET platform.

DOT NET TUTORIALS

Dot Net Tutorials

About the Author: Pranaya Rout

Pranaya Rout has published more than 3,000 articles in his 11-year career. Pranaya Rout has very good experience with Microsoft Technologies, Including C#, VB, ASP.NET MVC, ASP.NET Web API, EF, EF Core, ADO.NET, LINQ, SQL Server, MYSQL, Oracle, ASP.NET Core, Cloud Computing, Microservices, Design Patterns and still learning new technologies.